

Acta - UD6 Sesión 2

Profesor: Bastián Carnero Groba

UD6 — Servizos web con PHP e Flight

5 de maio de 2026

Resumo da sesión¹

Nesta segunda sesión da unidade afondamos na creación de servizos web con PHP e o microframework Flight. Partimos dun "Hola mundo" simple e fomos engadindo complexidade ata traballar con base de datos MySQL e operacións CRUD (GET, POST, DELETE).

Requisitos previos

- Ter MAMP instalado e funcionando (Apache + MySQL nos portos 8888 e 8889).
- Ter Visual Studio Code coa extensión Thunder Client.
- Ter o framework Flight descargado dentro da carpeta /Applications/MAMP/htdocs/ud06/flight.
- Ter o ficheiro .htaccess configurado na raíz do proxecto.

1. Hola mundo — Primeira ruta

Comezaremos co saúdo clásico da informática, creando estrutura básica dunha API que responde a peticións HTTP nunha URL concreta.

Código (index.php)

```
<?php
require 'flight/Flight.php';

Flight::route('/', function () {
    echo '¡Hola mundo desde mi primera API!';
});

Flight::start();
```

¹ O alumnado que non asistira pode poñerse ao día seguindo este documento de principio a fin. Cada apartado inclúe o código completo e as probas que se fixeron en clase con Thunder Client.

Proba

- Navegador: `http://localhost:8888/ud06/`
- Thunder Client: `GET http://localhost:8888/ud06/ → SEND`

Resultado esperado: `"¡Hola mundo desde mi primera API!"`

2. Máis rutas — Parámetros e JSON

Engadimos rutas con parámetros dinámicos na URL (`@nombre`) e a primeira ruta que devolve JSON estruturado, que é o que diferencia un servizo web dunha páxina convencional.

Código completo

```
<?php
require 'flight/Flight.php';

Flight::route('/', function () {
    echo '¡Hola mundo desde mi primera API!';
});

Flight::route('/saludar', function () {
    echo 'Hola, bienvenido al módulo de DWCS!';
});

Flight::route('/saludar/@nombre', function ($nombre) {
    echo "Hola, $nombre. ¿Qué tal estás?";
});

Flight::route('GET /productos', function () {
    $productos = [
        ['id' => 1, 'nombre' => 'Camiseta', 'precio' => 20],
        ['id' => 2, 'nombre' => 'Pantalón', 'precio' => 35],
        ['id' => 3, 'nombre' => 'Zapatos', 'precio' => 60]
    ];
    Flight::json($productos);
});

Flight::start();
```

Probos

- `http://localhost:8888/ud06/saludar`
- `http://localhost:8888/ud06/saludar/Maria`
- `http://localhost:8888/ud06/productos` (devolve JSON)

Concepto clave: a @ na URL indica que esa parte é variable. O valor que se poña aí pásase como argumento á función.

3. Conexión coa base de datos

Creamos a base de datos colegio en phpMyAdmin e conectámola á API. A partir deste momento, os datos xa non proceden código, senón da base de datos real.

3.1. Crear a base de datos

Acceder a phpMyAdmin: <http://localhost:8888/phpMyAdmin5/>

Na pestana SQL, executar:

```
CREATE DATABASE colegio;
USE colegio;

CREATE TABLE Modulo (
    id INT PRIMARY KEY,
    nombre VARCHAR(100),
    abreviatura VARCHAR(10)
);

INSERT INTO Modulo VALUES
(1, 'Desarrollo Web Cliente', 'DWEK'),
(2, 'Desarrollo Web Servidor', 'DWES'),
(3, 'Despliegue Aplicaciones Web', 'DAW'),
(4, 'Diseño de Interfaces Web', 'DIW');
```

3.2. Conectar a API á base de datos

Engadiuse ao index.php a conexión PDO e dúas rutas que len da BD:

```
Flight::register('db', 'PDO', array(
    'mysql:host=localhost;port=8889;dbname=colegio',
    'root',
    'root'
));

Flight::route('GET /modulos', function () {
    $sentencia = Flight::db()->prepare("SELECT * FROM Modulo");
    $sentencia->execute();
    $datos = $sentencia->fetchAll(PDO::FETCH_ASSOC);
    Flight::json($datos);
});

Flight::route('GET /modulos/@id', function ($id) {
    $sentencia = Flight::db()->prepare("SELECT * FROM Modulo WHERE id = :id");
    $sentencia->bindParam(':id', $id);
    $sentencia->execute();
    $dato = $sentencia->fetch(PDO::FETCH_ASSOC);
    Flight::json($dato);
});
```

Probas

- `http://localhost:8888/ud06/modulos` → todos os módulos
- `http://localhost:8888/ud06/modulos/2` → só o módulo 2

Pequena proba ilustrativa: na clase cambiamos un nome do módulo en phpMyAdmin (por exemplo de “Investigación” a “Investigación+Desarrollo”) e vimos como a API o reflicte automaticamente sen modificar o código PHP.

4. POST — Crear datos

Concepto clave: GET é para ler datos (parámetros na URL). POST é para crear datos (IMPORTANTE: os parámetros introdúcense en body, en formato JSON).

Como facer un POST en Thunder Client

- Cambiar o desplegable de GET a POST.
- Premer na pestana Body.
- Dentro de Body, escoller Json (NON Form, é o erro máis común).
- Pegar o JSON cos datos a enviar.
- Pulsar Send.

Exemplo de body JSON

```
{
  "a": 5,
  "b": 3
}
```

Resposta esperada:

```
{
  "a": 5,
  "b": 3,
  "resultado": 8
}
```

5. DELETE — Eliminar datos

Operación para borrar rexistros. Igual que POST, os datos van no body, non na URL.

Código completo (index.php)

```
<?php
require 'flight/Flight.php';
```

```

$GLOBALS['libros'] = [
    ['id' => 1, 'titulo' => 'El Quijote', 'autor' => 'Cervantes'],
    ['id' => 2, 'titulo' => 'Cien años de soledad', 'autor' => 'García Márquez'],
    ['id' => 3, 'titulo' => 'La sombra del viento', 'autor' => 'Ruiz Zafón'],
    ['id' => 4, 'titulo' => 'Los pilares de la Tierra', 'autor' => 'Ken Follett']
];

Flight::route('GET /libros', function () {
    Flight::json($GLOBALS['libros']);
});

Flight::route('DELETE /libros', function () {
    $id = Flight::request()->data->id;
    foreach ($GLOBALS['libros'] as $i => $libro) {
        if ($libro['id'] == $id) {
            unset($GLOBALS['libros'][$i]);
            Flight::json([
                'mensaje' => "Libro con id $id borrado correctamente",
                'libro_borrado' => $libro
            ]);
            return;
        }
    }
    Flight::json(['error' => "No existe ningún libro con id $id"], 404);
});

Flight::start();

```

Probas en Thunder Client

- GET <http://localhost:8888/ud06/libros> → ver os 4 libros.
- DELETE <http://localhost:8888/ud06/libros> con body { "id": 2 } → borra o libro 2.
- GET <http://localhost:8888/ud06/libros> → comprobar que xa só hai 3.
- DELETE con id que non existe (ex: 999) → devolve erro 404.

6. Tarefa final — API de Lista de tarefas

Como exercicio final, construímos unha API completa que combina todo o aprendido: GET con filtros, GET por id, POST para crear, e POST con acción para modificar. O código é:

```

<?php
require 'flight/Flight.php';

// "Base de datos" en memoria con tareas iniciales
$GLOBALS['tarefas'] = [
    ['id' => 1, 'titulo' => 'Estudiar para el examen', 'completada' => false],
    ['id' => 2, 'titulo' => 'Hacer la compra', 'completada' => true],
    ['id' => 3, 'titulo' => 'Llamar al dentista', 'completada' => false]
];

```

```

// =====
// GET - Ver todas las tareas
// =====
Flight::route('GET /tareas', function () {
    Flight::json($GLOBALS['tareas']);
});

// =====
// GET - Ver solo las pendientes
// =====
Flight::route('GET /tareas/pendientes', function () {
    $pendientes = [];
    foreach ($GLOBALS['tareas'] as $tarea) {
        if ($tarea['completada'] === false) {
            $pendientes[] = $tarea;
        }
    }
    Flight::json([
        'cantidad' => count($pendientes),
        'tareas' => $pendientes
    ]);
});

// =====
// GET - Ver solo las completadas
// =====
Flight::route('GET /tareas/completadas', function () {
    $completadas = [];
    foreach ($GLOBALS['tareas'] as $tarea) {
        if ($tarea['completada'] === true) {
            $completadas[] = $tarea;
        }
    }
    Flight::json([
        'cantidad' => count($completadas),
        'tareas' => $completadas
    ]);
});

// =====
// GET - Ver una tarea concreta por id
// =====
Flight::route('GET /tareas/@id', function ($id) {
    foreach ($GLOBALS['tareas'] as $tarea) {
        if ($tarea['id'] == $id) {
            Flight::json($tarea);
            return;
        }
    }
    // Si no la encontramos
    Flight::json(['error' => "No existe ninguna tarea con id $id"], 404);
});

// =====
// POST - Crear una tarea nueva
// =====

```

```

Flight::route('POST /tareas', function () {
  $titulo = Flight::request()->data->titulo;

  if (empty($titulo)) {
    Flight::json(['error' => 'El título es obligatorio'], 400);
    return;
  }

  // Calculamos el siguiente id
  $nuevo_id = count($GLOBALS['tareas']) + 1;

  $nueva_tarea = [
    'id'          => $nuevo_id,
    'titulo'      => $titulo,
    'completada' => false
  ];

  $GLOBALS['tareas'][] = $nueva_tarea;

  Flight::json([
    'mensaje' => 'Tarea creada correctamente',
    'tarea' => $nueva_tarea
  ], 201);
});

// =====
// POST - Marcar una tarea como completada
// =====
Flight::route('POST /tareas/@id/completar', function ($id) {
  foreach ($GLOBALS['tareas'] as $i => $tarea) {
    if ($tarea['id'] == $id) {
      $GLOBALS['tareas'][$i]['completada'] = true;
      Flight::json([
        'mensaje' => "Tarea $id marcada como completada",
        'tarea' => $GLOBALS['tareas'][$i]
      ]);
      return;
    }
  }
  Flight::json(['error' => "No existe ninguna tarea con id $id"], 404);
});

Flight::start();

```

Funcionalidades implementadas

- GET /tareas → ver todas as tarefas.
- GET /tareas/pendientes → só as non completadas.
- GET /tareas/completadas → só as completadas.
- GET /tareas/@id → unha tarefa concreta (devolve 404 se non existe).

- POST /tareas → crear unha tarefa nova (require título no body).
- POST /tareas/@id/completar → marcar unha tarefa como completada.

Estrutura de datos

```
$GLOBALS['tareas'] = [
  ['id' => 1, 'titulo' => 'Estudiar para el examen', 'completada' => false],
  ['id' => 2, 'titulo' => 'Hacer la compra', 'completada' => true],
  ['id' => 3, 'titulo' => 'Llamar al dentista', 'completada' => false]
];
```

Recorrido de probas

- GET /tareas → 3 tarefas iniciais.
- GET /tareas/pendientes → cantidad: 2.
- GET /tareas/2 → devolve só "Hacer la compra".
- POST /tareas con body { "titulo": "Sacar al perro" } → crea a tarefa 4.
- POST /tareas/1/completar → marca a tarefa 1 como completada.
- GET /tareas/pendientes → agora só hai 1 pendiente.

Conceptos clave da sesión

- Verbos HTTP: GET (ler), POST (crear), DELETE (borrar).
- Os parámetros pódense pasar pola URL (@nombre) ou polo body en formato JSON.

Reflexións

Modelos de xoguete

Os exemplos pareceron sinxelos, pero a estrutura é a mesma que usan apps moi populares:

- **A API de tarefas** → o esqueleto de Trello, Todoist ou Microsoft To Do.
- **A API de libros** → o que hai detrás de Goodreads ou da app da biblioteca.
- **A táboa Modulo** → unha versión minúscula de Aula Virtual, Moodle ou Google Classroom.
- **A calculadora con JSON** → mesma idea que as APIs de Google Maps, PayPal ou Spotify.

A diferenza é a escala (millóns de usuarios, miles de servidores), non o concepto. A idea fundamental é a aprendida hoxe.

A importancia de programar para os erros

Notade que en moitos exemplos programamos as mensaxes que o usuario recibirá se comete un erro.

Cómpre asumir que o usuario pode (e vai) equivocarse: escribirá letras onde se esperaba un número, pedirá algo que non existe, enviará formularios incompletos. Parte do traballo dun deseñador web é anticiparse a elo.

Unha API ben feita detecta o erro a tempo, devolve un código HTTP axeitado e un mensaxe comprensible. É o que fai Instagram cando rexeita unha foto grande, Gmail cando avisa de email mal escrito, ou o caixeiro cando di "*saldo insuficiente*" en lugar de quedar colgado.

Regra de ouro: *todo o que pode ir mal, vai ir mal algún día. Asegurádevos de que cando pase, a aplicación responda con elegancia.*